LP MVP Sdn Bhd.

The 2nd Round for The Mid-level Backend Position:

Take-home Assignment

Contents

1 Goals & Principles

Evaluation Criteria: Requirements fulfillment, architecture/layering, code quality, testing, security/robustness, performance/scalability, commit/PR hygiene.

Time Required: Approximately 7–8 hours total (can be split over multiple days).

Submission Deadline: 18:00PM, Thursday, 20 November 2025

Submission Email: parkchansu39@toncompany.co.kr

Autonomy: Clarifying questions are welcome anytime. Reasonable assumptions are fine if documented.

Open-source: Allowed with attribution. Add a short **CITATIONS.md** for libraries/articles you relied on.

2 What to Submit

Repository link (GitHub/GitLab), public or with access granted.

README: setup & run (local + Docker), decisions, trade-offs, time spent.

DB migrations & seeds (if applicable).

API documentation: OpenAPI/Swagger (YAML/JSON) or a Postman collection.

Tests: unit + integration/E2E (per role below).

Optional: demo URL.

Dockerfile and/or **docker-compose.yml**.

3 Domain Brief

Build a thin slice of a **payments-enabled order system**.

Suggested tables: users, products, orders, order_items, payments, refunds.

Example states - Order: PENDING → CONFIRMED → FULFILLED or CANCELLED - Payment: AUTHORIZED → CAPTURED or VOIDED or REFUNDED

Business rules (subset) - Store prices in minor units (e.g., cents). Currency: MYR. - **Idempotency**: Any POST that creates a payment must accept an Idempotency-Key header and behave idempotently within a 24h window. - **Reconciliation**: simple daily

totals by payment status/date.

Sample seed

```
{
  "products": [
    {"id": 1, "name": "Starter Plan", "price": 9900, "currency": "MYR"},
    {"id": 2, "name": "Pro Plan", "price": 19900, "currency": "MYR"}
  ]
}
```

4 Role-Specific Assignments

Tech: Laravel, MySQL, Redis, Docker.

Build

REST APIs - POST /orders (create order from product items) - GET /orders/{id} & GET /orders?userId= - POST /payments (authorize; **idempotent** via Idempotency-Key) - POST /payments/{id}/capture & POST /payments/{id}/void - POST /refunds (partial/whole)

Reconciliation endpoint - GET /reports/daily-settlement?date=YYYY-MM-DD

Persistence & design - Clean layering (controllers → services → repositories), DTOs/validators. - MySQL schema with appropriate indexes; avoid N+1.

Reliability - Background jobs/queues for capture/refund; document a DLQ strategy. - Structured logging (JSON) with correlation id (X-Request-ID).

Security - Input validation, rate-limit payment endpoints, secret management.

Tests - PEST/PHPUnit: unit (domain) and feature (API); cover 70%+ of payment paths.

Docker - docker-compose with app + MySQL + Redis; one-command startup.

Stretch (optional): Webhook simulator + signature verification; outbox pattern sketch; OpenAPI-generated client.

README must include: curl examples, how idempotency is implemented, and a

short scaling note (e.g., worker horizontals, DB partitioning candidates).

5 Fairness & Integrity

- Work must be your own. Cite external code or prompts in **CITATIONS.md**.
- Do not use proprietary code from past employers.
- Keep secrets out of the repo; use environment variables.

6 Candidate FAQ

- **How long?** Aim for the effort budget above within a 7-day window.
- **Can I ask questions?** Yes—**Whatsapp** anytime; assumptions are fine if documented.
- **Can I use libraries?** Yes; justify choices. Heavy UI kits are OK, but core logic must be yours.
- **What do you look at first?** README, tests, and how you handle errors/failures.